Randomized Fast Design of Short DNA Words*

Ming-Yang Kao † Manan Sanghi ‡ Robert Schweller §

February 18, 2005

Abstract

We consider the problem of efficiently designing sets (codes) of equal-length DNA strings that satisfy certain combinatorial constraints. This problem has numerous motivations including DNA computing and DNA self-assembly. Previous work has extended results from coding theory to obtain bounds on code size for new biologically motivated constraints and has applied heuristic local search and genetic algorithm techniques for code design. This paper proposes a natural optimization formulation of the DNA code design problem in which the goal is to design n strings that satisfy a given set of constraints while minimizing the length of the strings. For multiple sets of constraints, we provide high probability algorithms that run in time polynomial in n and any given constraint parameters, and output strings of length within a constant factor of the optimal. To the best of our knowledge, this work is the first to consider this type of optimization problem in the context of DNA code design.

1 Introduction

In this paper we study the problem of efficiently designing sets of DNA code words of near optimal length that fulfill certain combinatorial constraints. Many applications have emerged in recent years that depend on the scalable design of such words. One such problem is in DNA computing where inputs to computational problems are encoded into DNA strands for the purpose of computing via DNA complementary binding [1]. Another application involves implementing Wang tile self-assembly systems by encoding glues of Wang tiles into strands of DNA [18]. DNA codewords can also be used to store information at the molecular level [5], act as molecular bar codes for identifying molecules in complex libraries [5, 6, 14], or implement DNA arrays [3].

For a set of DNA code words to be effective for the above applications, they must fulfill a number of combinatorial constraints. Of particular importance is the need for specific hybridization between a given code word and its unique Watson-Crick complement. That is, we need to make sure that hybridization does not occur among a code word and the complement of a different word in the set, or even of any code word with any other code word in the set. For this requirement Marathe et al. [13] have proposed the Hamming constraint, reverse complement Hamming constraint, and self-complementary constraint. We further consider the more restricting shifting Hamming constraint which requires a large Hamming distance between all alignments of any pair of code words [7].

We also consider three constraints not related to Hamming distance. The consecutive base constraint limits the length of any run of identical bases in any given word. Long runs of identical

*Supported in part by NSF Grant EIA-0112934.

†Department of Computer Science, Northwestern University. Email: kao@cs.northwestern.edu.

‡Department of Computer Science, Northwestern University. Email: manan@cs.northwestern.edu.

§Department of Computer Science, Northwestern University. Email: schwellerr@cs.northwestern.edu.

Word Length and Time Complexity for DNA Word Design				
	Word Length		Time Complexity	
	Lower Bound	Upper Bound	Lower Bound	Upper Bound
$\mathrm{DWD}_{1,2,3,4,5,6}$	$\Theta(k+\log n)$ (Thm. 1)	$\Theta(k+\log n)$ (Thm. 3)	$\Theta(n\ell)$ (Thm. 1)	$\Theta(n\ell)$ (Thm. 3)
$\mathrm{DWD}_{1,2,5,6,7}$	$\Theta(k+\log n)$ (Thm. 1)	$\Theta(k+\log n)$ (Thm. 4)	$\Theta(n\ell)$ (Thm. 1)	$\Theta(n\ell)$ (Thm. 4)
$\mathrm{DWD}_{1,2,3,4,5,8}$	$\Theta(k+\log n)$ (Thm. 1)	$\Theta(k+\log n)$ (Thm. 6)	$\Omega(n\ell)$ (Thm. 1)	$O(\min(\ell^{1.5}\log^{0.5}\ell + n\ell, n\ell\log\ell))$ (Thm. 6)

Table 1: This table summarizes our results regarding the efficient design of DNA codeword sets. Here k denotes the maximum of the constraint parameters for constraints 1 through 5 (see Section 2). The value $\ell = \Theta(k + \log n)$ denotes the optimal achievable word length for the listed word design problems (Theorems 1, 3, 4 and 6).

bases is considered to cause hybridization errors [4, 7, 15]. The GC content constraint requires that a large percentage of the bases in any given word are either G or C. This constraint is meant to give each string similar thermodynamic properties [15–17]. The free energy constraint requires that the difference in free energy of any two strings is bounded by a small constant. This helps ensure that each string in the set has a similar melting temperature [7, 13].

In addition to the above constraints, it is desirable for the length ℓ of each code word to be as small as possible. The motivation for minimizing ℓ is evident from the fact that it is more difficult to synthesize longer strands. Similarly, longer strands require more DNA to be used for the respective application.

There has been much previous work in the design of DNA code words [5, 7, 10–14, 16, 17]. In particular, Marathe et al. [13] have extended results from coding theory to obtain bounds on code size for various biologically motivated constraints. However, most work in this area has been based on heuristics, genetic algorithms, and stochastic local searches that do not provide provably good results provably fast.

In this work we provide algorithms with analytical guarantees for combinatorial structures and time complexity. In particular, we formulate an optimization problem that takes as input a desired number of strings n and produces n length-ℓ strings that satisfy a specified set of constraints, while at the same time minimizing the length ℓ. We restrict our solution to this problem in two ways. First, we require that our algorithms run in time only polynomial in the number of strings n as well as any given constraint parameters. Second, we require that our algorithms produce sets of codewords that achieve word length ℓ that is within a constant multiple of the optimal achievable word length, while at the same time fulfilling the respective constraints with high probability. For various subsets of the constraints we propose, we provide algorithms that do this. We thus provide fast algorithms for the creation of sets of short codewords.

Paper Layout: In Section 2, we describe the different biologically motivated combinatorial constraints we use. In Section 3 we solve the design problem with subsets of constraints including the Hamming constraints, the consecutive bases constraint, and the GC content constraint. In Section 4 we extend our algorithms to deal with the free energy constraint.

2 Preliminaries

2.1 Notations

Let $X = x_1x_2 \dots x_\ell$ be a word where x_i belongs to some alphabet Π. In this paper we deal with two alphabets, namely, the binary alphabet $\Pi_B = \{0, 1\}$ and the DNA alphabet $\Pi_D = \{A, C, G, T\}$. The elements of an alphabet are called *characters*. We will use capital letters for words and small letters for characters.

The *reverse* of X, denoted by X^R, is the word $x_\ell x_{\ell-1} \dots x_1$. The *complement* of a character x is denoted by x^c and is defined as follows.

- $0^c = 1$, $1^c = O$;
- $A^c = T$, $C^c = G$, $G^c = C$, $T^c = A$.

The complement of a word is obtained by taking the complement of each of the characters in the word, i.e., $X^C = x_1^c x_2^c \dots x_\ell^c$. The *reverse complement* of X is the complement of X^R, $X^{RC} = x_\ell^c x_{\ell-1}^c \dots x_1^c$. The *Hamming distance* $H(X, Y)$ between two words X and Y is the number of positions where X differs from Y.

We are interested in designing a set $\mathcal{W}$ of n words each of length ℓ which satisfy the constraints defined in Section 2.2 below.

2.2 Constraints

The constraints we consider can be classified into categories: hybridization constraints and stability constraints. Hybridization constraints ensure that unwanted hybridizations between two strands are avoided, and stability constraints ensure that the strands are stable in a solution. The first five constraints below are hybridization constraint while the remaining three are stability constraints.

$C_1(k_1)$: **Hamming Constraint** (k_1)= For any codewords $Y, X \in \mathcal{W}$, $H(Y, X) \geq k_1$.

This constraint limits non-specific hybridizations between the Watson-Crick complement of some codeword Y with a distinct codeword X.

$C_2(k_2)$: **Reverse Complementary Constraint** (k_2)= For any codewords $Y, X \in \mathcal{W}$,

$$H(Y, X^{RC}) \geq k_2.$$

This constraint is intended to limit hybridization between a codeword and the reverse of another codeword.

$C_3(k_3)$: **Shifting Hamming Constraint** (k_3)= For any two codewords $Y, X \in \mathcal{W}$,

$$H(Y[1..i], X[(\ell - i + 1)..\ell]) \geq k_3 - (\ell - i) \text{ for all } i.$$

This is a stronger version of the Hamming Constraint.

$C_4(k_4)$: **Shifting Reverse Complementary Constraint** (k_4)= For any two codewords $Y, X \in \mathcal{W}$,

$$H(Y[1..i], X[(\ell - i + 1)..\ell]^{RC}) \geq k_4 - (\ell - i) \text{ for all } i.$$

This is a stronger version of the Reverse Complementary Constraint.

$C_5(k_5)$: **Self Complementary Constraint** (k_5)= For any codeword Y, $H(Y, Y^{RC}) \geq k_5$.

This constraint prevents a codeword from hybridizing with itself.

$C_6(\gamma)$: **GC Content Constraint** (γ)= γ percentage of bases in any codeword $Y \in \mathcal{W}$ should be either G or C.

The GC content affects the thermodynamic properties of a codeword [15–17]. Therefore, having the same ratio of GC content for all the codewords will assure similar thermodynamic characteristics.

$C_7(d)$: **Consecutive Base Constraint** (d)= No codeword should have more than d consecutive bases.

In some applications, consecutive occurrences (also known as runs) of the same base increase the number of annealing errors.

$C_8(\sigma)$: **Free Energy Constraint** (σ)= For any two codewords $Y, X \in \mathcal{W}$, $FE(Y) - FE(X) \leq \sigma$ where $FE(W)$ denotes the free energy of a word defined in Section 4.

This constraint ensures that all the codewords in the set have similar melting temperatures which allows hybridization of multiple strands to proceed simultaneously [14].

For each of the given constraints above we assign a shorthand boolean function $C_i(t)$ to denote whether or not a given set of words $\mathcal{W}$ fulfills constraint C_i with respect to parameter t. For a given integer n, the goal of DNA word design is to efficiently create a set of n length-ℓ words such that a given subset of the above constraints are satisfied, while trying to minimize ℓ. That is, for a given subset of constraints $\{C_{\pi_1}, C_{\pi_2}, \ldots, C_{\pi_r}\} \subseteq \{C_1, C_2, \ldots, C_8\}$, the corresponding DNA word design (DWD) optimization problem is as follows.

Problem 1 ($\text{DWD}_{\pi_1,\pi_2,\ldots,\pi_r}$)**.**
INPUT*: Integers* $n, t_1, t_2, \ldots, t_r$.
OUTPUT*: A set* $\mathcal{W}$ *of* n *strings each of the minimum length* ℓ *such that for all* $1 \leq i \leq r$ *the constraint* $C_{\pi_i}(t_i)$ *is satisfied over set* $\mathcal{W}$.

For this problem we have the following trivial lower bounds for time complexity and the size of ℓ when any one of the first five constraints is applied.

Theorem 1. *Consider a set of* n *length-*ℓ *words* $\mathcal{W}$.

1. *If* $\mathcal{W}$ *fulfills any one of the constraints* $C_1(k), C_2(k), C_3(k), C_4(k)$, *and* $C_5(k)$, *then* $\ell = \Omega(k + \log n)$.

2. *The time complexity of producing a set* $\mathcal{W}$ *that fulfills any one of the constraints* $C_1(k)$, $C_2(k)$, $C_3(k)$, $C_4(k)$, *and* $C_5(k)$ *is* $\Omega(nk + n \log n)$.

Algorithm $FastDWD_{1,2,3,4,5}(n, k_1, k_2, k_3, k_4, k_5)$

1. Let $k = \max\{k_1, k_2, k_3, k_4, k_5\}$.
2. Generate a set $\mathcal{W}$ of n words of length $\ell = 13 \cdot \max\{k, \log n\}$ uniformly at random.
3. Output $\mathcal{W}$.

Figure 1: Randomized algorithm for generating n strings satisfying constraints $C_1(k_1)$, $C_2(k_2)$, $C_3(k_3)$, $C_4(k_4)$, and $C_5(k_5)$.

PROOF: For any set of n distinct strings, ℓ is bounded below by $\log n$. Further, any two strings with a Hamming distance k should be each of length at least k. Thus $\ell \geq \max\{k, \log n\} = \Omega(k + \log n)$. Further, this gives an $\Omega(nk + n \log n)$ bound on the size of the output, yielding the lower bound on time complexity.■

The goal is to simultaneously satisfy as many of the above eight constraints as possible while achieving words within a constant factor of the optimal length ℓ for the given set of constraints. In Section 3 we show how to accomplish this goal for various subsets of the constraints.

3 Algorithms for DNA Word Design

In this section we develop randomized algorithms to generate sets of length-ℓ words that satisfy certain sets of constraints while keeping ℓ within a constant of the optimal value. In particular, we first show how simply generating a set of n words at a specific length $\ell = O(k + \log n)$ uniformly at random is sufficient to fulfill constraints 1, 2, 3, 4, and 5 simultaneously with high probability. We then propose three extensions to this algorithm to fulfill different subsets of constraints within a constant factor of the optimal word length. The first extension yields an algorithm for fulfilling the GC content constraint while the second yields one for the consecutive base constraint and the GC content constraint at the cost of shifting Hamming and shifting reverse complementary constraint. Finally, we extend the basic randomized scheme to fulfill the free energy constraint. The first is thus an algorithm for simultaneously fulfilling constraints 1, 2, 3, 4, 5, and 6, the second simultaneously fulfills constraints 1, 2, 5, 6, and 7, and the last one fulfills constraints 1, 2, 3, 4, 5, and 8.

3.1 A Simple Randomized Algorithm

Problem 2 ($\text{DWD}_{1,2,3,4,5}$)**.**
INPUT*: Integers n, k_1, k_2, k_3, k_4, k_5.*
OUTPUT*: A set $\mathcal{W}$ of n strings each of the minimum length ℓ such that the constraints $C_1(k_1)$, $C_2(k_2)$, $C_3(k_3)$, $C_4(k_4)$, $C_5(k_5)$ hold.*

The next theorem shows that Algorithm $\text{FastDWD}_{1,2,3,4,5}(n, k_1, k_2, k_3, k_4, k_5)$ in Figure 1 yields a polynomial time solution to the $DWD_{1,2,3,4,5}$ problem with high probability.

Theorem 2. *Algorithm* $\text{FastDWD}_{1,2,3,4,5}$ *produces a set $\mathcal{W}$ of n words of length $\ell = \Theta(k + \log n)$ in time $O(n \cdot k + n \cdot \log n)$ satisfying constraints $C_1(k_1)$, $C_2(k_2)$, $C_3(k_3)$, $C_4(k_4)$, and $C_5(k_5)$ with probability of failure at most $o(1/(n + 2^k))$, where $k = \max\{k_1, k_2, k_3, k_4, k_5\}$.*

PROOF: We show that $\mathcal{W}$ satisfies constraint $C_i(k)$ for all $1 \leq i \leq 5$ with high probability. Since $k \geq k_i$, this immediately implies that $\mathcal{W}$ satisfies $C_i(k_i)$.

Let $k^* = \max\{k, \log n\}$ and $\ell = ck^*$ for some constant c. We will choose an appropriate c below. Let S be the size of the alphabet. Let $P_i(Y, X)$ for all $1 \leq i \leq 4$ be the probability that constraint $C_i(k^*)$ is violated by strings Y and X generated uniformly at random. Let $P_5(Y)$ be the probability that a randomly generated Y violate constraint $C_5(k^*)$.

$$
\begin{aligned}
P_1(Y,X) &= \Pr\big[H(Y,X) < k^*\big] \\
&= \sum_{x=0}^{k^*-1} \binom{\ell}{x} \frac{(S-1)^x}{S^\ell} \\
&\leq \frac{1-\frac{k^*}{\ell}}{1-\frac{k^*S}{\ell(S-1)}} \binom{\ell}{k^*} \frac{(S-1)^{k^*}}{S^\ell} \\
&= \frac{(S-1)c-(S-1)}{(S-1)c-S} \binom{\ell}{k^*} \frac{(S-1)^{k^*}}{S^\ell} \\
&\leq \left(\frac{(S-1)c-(S-1)}{(S-1)c-S}\right)\left(\frac{\ell e}{k^*}\right)^{k^*}\left(\frac{(S-1)^{k^*}}{S^\ell}\right) \\
&= \left(\frac{(S-1)c-(S-1)}{(S-1)c-S}\right)\left(ce\right)^{k^*}\left(\frac{(S-1)}{S^c}\right)^{k^*} \\
&= \left(\frac{(S-1)c-(S-1)}{(S-1)c-S}\right)\left(\frac{(S-1)ce}{S^c}\right)^{k^*} \\
&= \left(\frac{(S-1)c-(S-1)}{(S-1)c-S}\right)\left(S^{k^*}\right)^{\log_S \frac{(S-1)ce}{S^c}} \\
&< 2(S^{k^*})^{\log_S \frac{(S-1)ce}{S^c}} \qquad (\text{for } c > \tfrac{S+1}{S-1}).
\end{aligned}
$$

Similarly, since $\Pr[H(Y, X^{RC}) < k^*] = \sum_{x=0}^{k^*-1} \binom{\ell}{x} \frac{(S-1)^x}{S^\ell}$, we get $P_2(Y,X) < 2(S^{k^*})^{\log_S \frac{(S-1)ce}{S^c}}$.

$$
\begin{aligned}
P_3(Y,X) &= \Pr\Big[\bigcup_i H(Y[1..i], X[(\ell-i+1)..\ell]) < k^* - (\ell - i)\Big] \\
&\leq \sum_{i=\ell-k^*+1}^{\ell} \Pr\Big[H(Y[1..i], X[(\ell-i+1)..\ell]) < k^* - (\ell-i)\Big] \\
&= \sum_{i=\ell-k^*+1}^{\ell} \Pr\Big[B(i, (S-1)/S) < k^* - \ell + i\Big] \\
&\leq \sum_{i=\ell-k^*+1}^{\ell} \frac{(S-1)(i-(k^*-\ell+i))}{(S-1)i - S(k^*-\ell+i)} \binom{i}{k^*-(\ell-i)} \frac{(S-1)^{i-(l-k^*)}}{S^i}
\end{aligned}
$$

V

$$\begin{aligned}
P_3(Y,X) &= \sum_{i=\ell-k^*+1}^{\ell} \frac{(S-1)(\ell-k^*)}{S(\ell-k^*)-i}\binom{i}{\ell-k^*}\frac{(S-1)^{i-(l-k^*)}}{2^i} \\
&\leq \frac{(S-1)(\ell-k^*)}{(S-1)\ell-Sk^*}\binom{\ell}{\ell-k^*}\frac{(S-1)^{k^*}}{S^{\ell-k^*+1}} \\
&< \left(\frac{(S-1)(c-1)}{(S-1)c-S}\right)\left(\frac{\ell e}{k^*}\right)^{k^*}\frac{(S-1)^{k^*}}{S^{\ell-k^*}} \\
&= \left(\frac{(S-1)(c-1)}{(S-1)c-S}\right)\left(\frac{(S-1)ce}{S^{c-1}}\right)^{k^*} \\
&= \left(\frac{(S-1)(c-1)}{(S-1)c-S}\right)\left(S^{k^*}\right)^{\log_S \frac{S(S-1)ce}{S^c}} \\
&< 2(S^{k^*})^{\log_S \frac{S(S-1)ce}{S^c}} \qquad (\text{for } c > \tfrac{S+1}{S-1}).
\end{aligned}$$

Similarly, we also get $P_4(Y,X) < 2(S^{k^*})^{\log_S \frac{S(S-1)ce}{S^c}}$.

$$\begin{aligned}
P_5(Y) &= \Pr\big[H(Y,Y^{RC}) < k^*\big] \\
&= \sum_{x=0}^{\frac{k^*-1}{2}} \binom{\frac{\ell}{2}}{x}\frac{(S-1)^x}{S^{\frac{\ell}{2}}} \\
&\leq \frac{1-\frac{k^*}{\ell}}{1-\frac{k^*S}{\ell*(S-1)}}\binom{\frac{\ell}{2}}{k^*}\frac{(S-1)^{k^*}}{S^{\ell}} \\
&= \frac{(S-1)(c-1)}{3(S-1)-S}\binom{\frac{\ell}{2}}{\frac{k^*}{2}}\frac{3^{\frac{k^*}{2}}}{S^{\frac{\ell}{2}}} \\
&\leq \left(\frac{(S-1)(c-1)}{(S-1)c-S}\right)\left(\frac{\ell e}{k^*}\right)^{\frac{k^*}{2}}\left(\frac{(S-1)^{\frac{k^*}{2}}}{S^{\frac{\ell}{2}}}\right) \\
&= \left(\frac{(S-1)(c-1)}{(S-1)c-S}\right)\left(ce\right)^{\frac{k^*}{2}}\left(\frac{S-1}{S^c}\right)^{\frac{k^*}{2}} \\
&= \left(\frac{(S-1)(c-1)}{(S-1)c-S}\right)\left(\frac{(S-1)ce}{S^c}\right)^{\frac{k^*}{2}} \\
&= \left(\frac{(S-1)(c-1)}{(S-1)c-S}\right)\left(S^{k^*}\right)^{(\log_S \frac{(S-1)ce}{S^c})/2} \\
&< 2(S^{k^*})^{(\log_S \frac{(S-1)ce}{S^c})/2} \qquad (\text{for } c > \tfrac{S+1}{S-1}).
\end{aligned}$$

The probability that $\mathcal{W}$ does not satisfy constraint C_i is at most $\binom{n}{2}P_i(Y,X) < \frac{n^2}{2}P_i(Y,X)$ for all $1 \leq i \leq 4$. Similarly, the probability that $\mathcal{W}$ does not satisfy constraint C_5 is at most $nP_5(Y)$. Therefore for $c > (S+1)/(S-1)$, the probability that $\mathcal{W}$ does not satisfy any of the five constraints is at most $\sum_{i=1}^{4} n^2 P_i(Y,X) + nP_5(Y) = 2(S^{k^*})^{\log_S \frac{(S-1)ce}{S^c}+2} + 2(S^{k^*})^{\log_S \frac{S(S-1)ce}{S^c}+2} + 2(S^{k^*})^{(\log_S \frac{(S-1)ce}{S^c}+2)/2} \leq 6n^{1+(\log_S \frac{(S-1)ce}{S^c}+2)/2}$ which is at most $o(1/2^{k^*}) = O(1/(n+2^k))$ for $c \geq 13$. ■

Algorithm $FastDWD_{1,2,3,4,5,6}(n, k_1, k_2, k_3, k_4, k_5, \gamma)$

1. Let $k = \max\{k_1, k_2, k_3, k_4, k_5\}$.
2. Generate a set $\mathcal{W}$ of n words over the binary alphabet Π_B of length $\ell = 13 \cdot \max\{k, \log n\}$ uniformly at random.
3. For each word $W \in \mathcal{W}$, for any $\gamma \cdot \ell$ characters in Y, replace 0 by G and 1 by C. For the remaining characters replace 0 by A and 1 by T to get W'. Let $\mathcal{W}'$ be the set of all words W'.
4. Output $\mathcal{W}'$.

Figure 2: Algorithm for generating n strings satisfying constraints $C_1(k_1)$, $C_2(k_2)$, $C_3(k_3)$, $C_4(k_4)$, $C_5(k_5)$, and $C_6(\gamma)$.

3.2 Incorporating the GC Content Constraint into FastDWD$_{1,2,3,4,5}$

Now we show how to modify Algorithm FastDWD$_{1,2,3,4,5}$ so that it produces a set that also satisfies the GC content constraint. That is, we will show how to solve the following problem.

Problem 3 (DWD$_{1,2,3,4,5,6}$)**.**
INPUT*: Integers n, k_1, k_2, k_3, k_4, k_5, γ.*
OUTPUT*: A set $\mathcal{W}$ of n strings each of the minimum length ℓ such that the constraints $C_1(k_1)$, $C_2(k_2)$, $C_3(k_3)$, $C_4(k_4)$, $C_5(k_5)$, $C_6(\gamma)$ hold.*

We modify Algorithm FastDWD$_{1,2,3,4,5}$ to get Algorithm FastDWD$_{1,2,3,4,5,6}$ shown in Figure 2. The next theorem shows that FastDWD$_{1,2,3,4,5,6}$ yields a polynomial time solution to $DWD_{1,2,3,4,5,6}$ with high probability.

Theorem 3. *Algorithm* FastDWD$_{1,2,3,4,5,6}$ *produces a set $\mathcal{W}$ of n words of length $\ell = \Theta(k + \log n)$ in time $O(n \cdot k + n \cdot \log n)$ satisfying constraints $C_1(k_1)$, $C_2(k_2)$, $C_3(k_3)$, $C_4(k_4)$, $C_5(k_5)$, and $C_6(\gamma)$ with probability of failure at most $o(1/(n + 2^k))$, where $k = \max\{k_1, k_2, k_3, k_4, k_5\}$.*

PROOF: From Theorem 2, $\mathcal{W}$ satisfies constraints $C_1(k_1)$, $C_2(k_2)$, $C_3(k_3)$, $C_4(k_4)$, and $C_5(k_5)$ with high probability. The construction of words in $\mathcal{W}'$ ensures that if two words $X, Y \in \mathcal{W}$ satisfy constraints $C_1(k_1)$, $C_2(k_2)$, $C_3(k_3)$, $C_4(k_4)$, and $C_5(k_5)$, then the corresponding words $X', Y' \in \mathcal{W}'$ satisfy them too. In addition, all words in $\mathcal{W}'$ satisfy $C_6(\gamma)$ with probability 1. ■

3.3 Incorporating the Consecutive Bases Constraint into FastDWD$_{1,2,3,4,5,6}$

Now we modify Algorithm FastDWD$_{1,2,3,4,5,6}$ so that it produces a set that satisfies both the GC content and consecutive base constraints at the cost of shifting hamming and shifting reverse complementary constraints. That is, we will show how to solve the following problem.

Problem 4 (DWD$_{1,2,5,6,7}$)**.**
INPUT*: Integers n, k_1, k_2, k_5, γ, d.*
OUTPUT*: A set $\mathcal{W}$ of n strings each of the minimum length ℓ such that the constraints $C_1(k_1)$, $C_2(k_2)$, $C_5(k_5)$, $C_6(\gamma)$, $C_7(d)$ hold.*

Algorithm *BreakRuns*(X, d)

1. Let $X = x_1 x_2 \dots x_\ell$. For $0 < i \leq \lceil \frac{\ell}{2(d-1)} \rceil - 1$, let $x'_{li} = x^c_{i(d-1)}$ and $x'_{ri} = x^c_{\ell - i(d-1)}$. Let $x'_{\text{mid}} = x^c_{\lfloor \ell/2 \rfloor}$.
2. Output $X' = x_1 \dots x_{d-1} x'_{l1} x_d \dots x_{\lfloor \ell/2 \rfloor} x'_{\text{mid}} x_{\lfloor \ell/2 \rfloor + 1} \dots x_{\ell-(d-1)-1} x'_{r1} x_{\ell-(d-1)} \dots x_\ell$.

Algorithm FastDWD$_{1,2,5,6,7}(n, k_1, k_2, k_5, \gamma, d)$

1. Let $k = \max\{k_1, k_2, k_5\}$.
2. Generate a set $\mathcal{W}$ of n words over the binary alphabet Π_B of length $\ell = 13 \cdot \max\{k, \log n\}$ uniformly at random.
3. For each word $W \in \mathcal{W}$, let $W' = \text{BreakRuns}(Y, d)$. Let $\mathcal{W}'$ be the set of all words W'.
4. For each word $W' \in \mathcal{W}'$, for any $\gamma\ell$ characters in W', replace 0 by G and 1 by C. For the remaining characters replace 0 by A and 1 by T to get W''. Let $\mathcal{W}''$ be the set of all words W''.
5. Output $\mathcal{W}''$.

Figure 3: Algorithms for generating n strings satisfying constraints $C_1(k_1)$, $C_2(k_2)$, $C_5(k_5)$, $C_6(\gamma)$, and $C_7(d)$.

We use Algorithm BreakRuns shown in Figure 3 to break long runs for a binary word so that it satisfies the consecutive bases constraint with parameter d. Intuitively what this algorithm does is for a given word X, it outputs X' by inserting characters at intervals of $d-1$ from the left and the right in a manner such that there are no consecutive runs of length greater than d. We need to add characters from both the ends to ensure that $H(X, Y^{RC}) \leq H(X', Y'^{RC})$ where X' and Y' are the respective outputs for X and Y.

We modify Algorithm FastDWD$_{1,2,3,4,5,6}$ to get Algorithm FastDWD$_{1,2,5,6,7}$ shown in Figure 3. The next theorem shows that FastDWD$_{1,2,5,6,7}$ yields a polynomial time solution to DWD$_{1,2,5,6,7}$ with high probability.

Theorem 4. *Algorithm* FastDWD$_{1,2,5,6,7}$ *produces a set* $\mathcal{W}$ *of* n *words of length* $\ell = \Theta(k + \log n)$ *in time* $O(n \cdot k + n \cdot \log n)$ *satisfying constraints* $C_1(k_1)$, $C_2(k_2)$, , $C_5(k_5)$, $C_6(\gamma)$, *and* $C_7(d)$ *with probability of failure at most* $o(1/(n + 2^k))$, *where* $k = \max\{k_1, k_2, k_5\}$.

Proof: For any two strings X, Y of length ℓ, let $X' = \text{BreakRuns}(X, d)$ and $Y' = \text{BreakRuns}(Y, d)$. Observe the following inequalities.

1. $H(X, Y) \leq H(X', Y')$;
2. $H(X, Y^{RC}) \leq H(X', Y'^{RC})$;
3. $H(X, X^{RC}) \leq H(X', X'^{RC})$; and
4. The length of X'(or Y') is at most $\ell(1 + \frac{1}{d-1}) < 2\ell$.

Algorithm $FastDWD_{1,2,3,4,5,8}(n, k_1, k_2, k_3, k_4, k_5)$

1. Let $k = \max\{k_1, k_2, k_3, k_4, k_5\}$.
2. Generate a set $\mathcal{W}$ of n words of length $\ell = 13 \cdot \max\{k, \log n\}$ uniformly at random.
3. If $FE(S_b) - FE(S_a) \leq 3D$, then output $\mathcal{W}$.
4. Let $\alpha = FE(S_b) + S'_1$ and $\beta = \alpha + \Delta$. For each $S_i \in \mathcal{W}$, find S'_j such that $\alpha \leq FE(S_i) + FE(S'_j) \leq \beta$. Let $W'_i = S_i \otimes S'_j$.
5. Output $\mathcal{W}' = \{W_1, \ldots, W_n\}$.

Figure 4: Algorithm for generating n strings satisfying constraints $C_1(k_1)$, $C_2(k_2)$, $C_3(k_3)$, $C_4(k_4)$, $C_5(k_5)$, and $C_8(4D + \Gamma_{\max})$.

From Theorem 2, $\mathcal{W}$ satisfies the constraints $C_1(k_1)$, $C_2(k_2)$, and $C_5(k_5)$ with probability of failure at most $o(1/(n + 2^k))$. Thus, the first three observations imply that $\mathcal{W}''$ will also satisfy these constraints in addition to the constraints $C_8(d)$ and $C_6(\gamma)$ with probability of failure at most $o(1/(n + 2^k))$. Further, from the last observation we know that the length of words in $\mathcal{W}$ increases by at most a constant factor. ■

4 Incorporating the Free Energy Constraint into FastDWD$_{1,2,3,4,5}$

Now we give an alternate modification of Algorithm FastDWD$_{1,2,3,4,5}$ can be made so that the free energy constraint is satisfied. The free-energy of a DNA word (strand) $X = x_1 x_2 \ldots x_\ell$ is approximated by the following formula from Breslauer et al. [8].

$$\text{FE}(X) = \text{correction factor} + \sum_{i=1}^{\ell-1} \Gamma_{x_i, x_{i+1}},$$

where $\Gamma_{x,y}$ is the pairwise free energy between base x and base y. For simplicity, we denote the free energy as simply the sum $\sum_{i=1}^{\ell-1} \Gamma_{x_i, x_{i+1}}$ with respect to a given pairwise energy function Γ. Let $\Gamma_{\max}$ and $\Gamma_{\min}$ be the maximum and the minimum entries in Γ respectively. Let $D = \Gamma_{\max} - \Gamma_{\min}$.

We now show how to satisfy the free energy constraint $C_8(\sigma)$ for a constant $\sigma = 4D + \Gamma_{\max}$, while simultaneously satisfying constraints $1, 2, 3, 4$, and 5. That is, we show how to solve the following problem.

Problem 5 (DWD$_{1,2,3,4,5,8}$)**.**
INPUT*: Integers n, k_1, k_2, k_3, k_4, k_5.*
OUTPUT*: A set $\mathcal{W}$ of n strings each of the minimum length ℓ such that the constraints $C_1(k_1)$, $C_2(k_2)$, $C_3(k_3)$, $C_4(k_4)$, $C_5(k_5)$, $C_8(4D + \Gamma_{\max})$ hold.*

To solve this problem we introduce the following notations. Let $S'_1, S'_2, \ldots, S'_{4^m}$ be all possible sequences of length $m = 2\ell$ such that $\text{FE}(S'_1) \leq \text{FE}(S'_2) \leq \cdots \leq \text{FE}(S'_{4^m})$. Let $\Delta = \max_i\{\text{FE}(S'_{i+1}) - \text{FE}(S'_i)\}$. For two strings X and Y of lengths a and b respectively, let $X \otimes Y$ be the string $Y[1..(b/2)]\ X[1..a]\ Y[(b/2+1)..b]$. Let $a = \arg\min_i\{\text{FE}(S_i)\}$ and $b = \arg\max_i\{\text{FE}(S_i)\}$.

We modify Algorithm FastDWD$_{1,2,3,4,5}$ to get Algorithm FastDWD$_{1,2,3,4,5,8}$ shown in Figure 4 for solving $DWD_{1,2,3,4,5,8}$. In Section 4.1 we give some lemmas useful for proving the correctness of Algorithm FastDWD$_{1,2,3,4,5,8}$. Section 4.2 discusses the details for Step 4 of the algorithm. Finally, in Section 4.3 we establish its correctness and time complexity.

4.1 Lemmas for the Correctness of FastDWD$_{1,2,3,4,5,8}$

Lemma 1. *$\Delta < 2D$.*

PROOF: For any string W of length m such that $FE(W) < FE(S'_{4^m})$, we construct a string W_{i^*} such that $FE(W) < FE(W_{i^*}) \leq FE(W)+2D$. Let W' be any string such that $FE(W') > FE(W)$ and let q be the Hamming distance between W and W'. Let W_i denote a string generated by changing the first i characters where W differed from W' such that Hamming distance between W_i and W' is $q - i$. Since changing one character in a string can increase the free energy by at most $2D$, $FE(W_i) \leq FE(W_{i-1}) + 2D$. Consider the smallest i^* such that $FE(W_{i^*}) > FE(W_i)$. Since $FE(W_{i^*-1}) \leq FE(W)$, we have $FE(W_{i^*}) \leq FE(W_{i^*-1}) + 2D \leq FE(W) + 2D$. ■

Lemma 2. *If $FE(S_b) - FE(S_a) > 3D$, then $FE(S_b) - FE(S_a) + 2D \leq FE(S'_{4^m}) - FE(S'_1)$.*

PROOF:

$$\begin{aligned} FE(S'_{4^m}) &\geq 2FE(S_b) + \Gamma_{\min} \\ FE(S'_1) &\leq 2FE(S_a) + \Gamma_{\max} \\ FE(S'_{4^m}) - FE(S'_1) &\geq 2(FE(S_b) - FE(S_a)) + \Gamma_{\min} - \Gamma_{\max} \\ FE(S'_{4^m}) - FE(S'_1) &\geq FE(S_b) - FE(S_a) + 2D. \end{aligned}$$

■

Lemma 3. *For each $S_i \in \mathcal{W}$, there exists S'_j such that $\alpha \leq FE(S_i) + FE(S'_j) \leq \beta$.*

PROOF:

$$\begin{aligned} FE(S_b) - FE(S_a) + 2D &\leq FE(S'_{4^m}) - FE(S'_1) \qquad \text{(Lemma 2)} \\ FE(S_b) + FE(S'_1) + 2D &\leq FE(S'_{4^m}) + FE(S_a) \\ FE(S_i) + FE(S'_1) &\leq FE(S_b) + FE(S'_1) = \alpha \\ FE(S_i) + FE(S'_{4^m}) &\geq FE(S_a) + FE(S'_{4^m}) \geq \alpha + 2D. \end{aligned}$$

Therefore, since $\Delta < 2D$, there exists S'_j such that $\alpha \leq FE(S_i) + FE(S'_j) \leq \beta$. ■

Lemma 4. *For all i, $\alpha - D \leq FE(T_i) \leq \beta + D + \Gamma_{\max}$.*

PROOF: Since $\alpha \leq FE(S_i) + FE(S'_j) \leq \beta$, $\alpha - D \leq FE(S_i \otimes S'_j) \leq \beta + D + \Gamma_{\max}$. ■

4.2 Computing Energy E Strings

In Step 4 of Algorithm FastDWD$_{1,2,3,4,5,8}$ we need to produce a set of n strings $S'_1, S'_2, \ldots S'_n$, each of a given length $L = m$, such that $A_i \leq FE(S'_i) \leq B_i$ for some A_i, B_i such that $B_i - A_i \leq \Delta$. That is, we need to solve the following problem.

Problem 6 (Bounded Energy Strand Generation Problem).
INPUT*:*

1. *Integers A_i and B_i for $i = 1$ to n such that $B_i - A_i \leq \Delta$.*

2. *Length L.*

OUTPUT: *Strings $S'_1, S'_2, \ldots S'_n$ each of length L and respective energy E_i such that $A_i \leq E_i \leq B_i$.*

Our solution to this problem involves transforming the blunt of the computational task into the problem of polynomial multiplication. Consider the following polynomial.

Definition 1.

1. *For any integer $\ell \geq 1$, let $f_{\ell,a,b}(x)$ be the polynomial $\sum_{e=0}^{\ell \cdot m} \zeta_e x^e$ where coefficient ζ_e is the number of length ℓ strings whose first character is a, last character is b, and free energy is e.*

2. *For even ℓ, let $f_{\ell,a,b}^{d_1,d_2}(x)$ denote the polynomial whose coefficients ζ_e are the number of length ℓ and energy e strings with a as the first character, b as the last, d_1 as the $\frac{\ell}{2}$ character, and d_2 and the $\frac{\ell}{2}+1$ character.*

3. *For odd $\ell > 1$, let $f_{\ell,a,b}^{d_1,d_3,d_2}$ denote the polynomial whose coefficients ζ_e are the number of strings of length ℓ and energy e that have a for the first character, b for the last, d_1 for the $\lfloor \frac{\ell}{2} \rfloor$ character, d_3 for the $\lfloor \frac{\ell}{2} \rfloor + 1$ character, and d_2 for the $\lfloor \frac{\ell}{2} \rfloor + 2$ character. For convenience, define $f_{1,a,b}^{d_1,d_3,d_2}(x)$ to be NULL.*

For $f_\ell(x) = \sum_{\forall a,b \in \Pi} f_{\ell,a,b}(x)$ the coefficient of x^i denotes the number of strings of length ℓ and free energy i. As a first step towards our solution, we present a subroutine BUILD(L) which computes the polynomials $f_{L,a,b}(x), f_{\lfloor L/2 \rfloor,a,b}(x), \ldots, f_{1,a,b}(x)$, for all $a, b \in \Pi$ in $O(L \log L)$. The problem of determining the number of strings of length L and free energy E is considered in [13] and a $O(L^2)$ solution is provided. As shown in Lemma 6 below our subroutine solves this problem in a faster $O(L \log L)$ time and may be of independent interest.

Our algorithm has two phases, the *build* phase and the *extract* phase. The build phase (BUILD(L) and SlowBuild(L) shown in Figures 6 and 8 respectively) constructs a data structure that permits the extract phase to be executed quickly. In the extract phase, an extraction routine (Extract(E) shown in Figure 7) is run n times to output S'_i for each $i \in [1, n]$. Since the extraction routine is executed n times and the build routine only once, the phase that constitutes the bottleneck for the entire procedure depends on the value of n. We thus provide two forks for the algorithm to take, one with a fast build routine and a modestly fast extract routine, and another with a slower build routine but an optimally fast extract routine. In particular, if n is sufficiently larger than L, the algorithm calls a routine SlowBuild(L) which improves the runtime of Extract. Otherwise, only a faster BUILD function is called in the first phase, leading to a slower Extract routine. The algorithm is given in Figure 5.

To describe the three subroutines of Algorithm ConstructStrings we use the following notation. Let $S = |\Pi|$ be the size of the alphabet of interest ($S = 4$ for DNA bases). Denote the largest pairwise energy $\Gamma_{\max}$ by M. For polynomial P, let ζ_e^P denote the coefficient of x^e in P.

The procedure Build(L) computes for all $a, b \in \Pi$ and a given L, the polynomials $f_{\ell,a,b}(x)$ for $\ell = L, \lfloor \frac{L}{2} \rfloor, \lfloor \frac{L}{4} \rfloor, \lfloor \frac{L}{8} \rfloor, \ldots, 1$. The efficient computation of these polynomials relies on the recursive property in Lemma 5 below.

Lemma 5. *For any integers $\ell_1, \ell_2 \geq 1$,*

$$f_{\ell_1+\ell_2,a,b}(x) = \sum_{d_1,d_2 \in \Pi} f_{\ell_1,a,d_1}(x) \cdot f_{\ell_2,d_2,b}(x) \cdot x^{\Gamma_{d_1,d_2}}.$$

Algorithm *ConstructStrings*$(\{A_i\}, \{B_i\}, L)$

1. Let $\Phi \leftarrow$ Build(L).
2. If $n \geq \sqrt{\frac{L}{\log L}}$, then $\Psi \leftarrow$ SlowBuild(L), else $\Psi \leftarrow$ NULL.
3. For each $i = 1$ to n, find a non-zero coefficient ζ_{E_i} of X^{E_i} in polynomial F_L such that $A_i \leq E_i \leq B_i$.
4. For $i = 1$ to n, set $S'_i =$Extract(E_i, Φ, Ψ).

Figure 5: This algorithm solves the Bounded Energy Strand Generation Problem (Problem 6).

PROOF: For any given integer $e \geq 0$, the coefficient of $x^{e-\Gamma_{d_1,d_2}}$ in polynomial $f_{\ell_1,a,d_1}(x) \cdot f_{\ell_2,d_2,b}(x)$ equals the number of ways to pick a length ℓ_1 string of some energy e_1 and a length ℓ_2 string of some energy e_2 such that $e_1 + e_2 = e - \Gamma_{d_1,d_2}$. This value is the number of length ℓ strings of energy e such that positions ℓ_1 and $\ell_1 + 1$ are characters d_1 and d_2 respectively. By multiplying the polynomial by $x^{\Gamma_{d_1,d_2}}$ this value becomes the coefficient of x^e. By summing over all $d_1, d_2 \in \Pi$ we get the coefficient of x^e to be the total number of strings of energy e for any d_1, d_2. ■

We now show the correctness and time complexity of Build(L) given in Figure 6.

Lemma 6 (Correctness and Time Complexity of Build(ℓ)).

1. *Build(ℓ) fulfills the output specifications given in Figure 6.*
2. *Build(ℓ) runs in $O(\ell \log \ell)$ time.*

PROOF:

Statement 1. To show the correctness of Build(L) for an arbitrary L, it suffices to show that $\hat{f}_{\lfloor \frac{L}{2^i} \rfloor,a,b}(x) = f_{\lfloor \frac{L}{2^i} \rfloor,a,b}(x)$, $\hat{f}^{d_1,d_2}_{\lfloor \frac{L}{2^i} \rfloor,a,b}(x) = f^{d_1,d_2}_{\lfloor \frac{L}{2^i} \rfloor,a,b}(x)$, and $\hat{f}^{d_1,d_3,d_2}_{\lfloor \frac{L}{2^i} \rfloor,a,b}(x) = f^{d_1,d_3,d_2}_{\lfloor \frac{L}{2^i} \rfloor,a,b}(x)$ for $i = 0$ to $\lfloor \log L \rfloor$. By induction on L, we show that $\hat{f}_{\lfloor \frac{L}{2^i} \rfloor,a,b}(x) = f_{\lfloor \frac{L}{2^i} \rfloor,a,b}(x)$. The proof for the remaining two equalities is essentially the same. As a base case, for $L = 1$ the equality clearly holds from Step 1. Inductively, assume the correctness of Build$(\lfloor \frac{L}{2} \rfloor)$. We thus have that $\hat{f}_{\ell,a,b} = f_{\ell,a,b}$ for all $a, b \in \Pi$ and $\ell = \lfloor \frac{\lfloor \frac{L}{2} \rfloor}{2^i} \rfloor$ for $i = 0, \ldots \lfloor \log \lfloor \frac{L}{2} \rfloor \rfloor$. For each i we have $\lfloor \frac{\lfloor \frac{L}{2} \rfloor}{2^i} \rfloor = \lfloor \frac{L}{2^{i+1}} \rfloor$. Thus, it only remains to show that $\hat{f}_{L,a,b} = f_{L,a,b}$. For even L, from Lemma 5 and the fact that $\hat{f}_{\lfloor \frac{L}{2} \rfloor,a,b} = f_{\lfloor \frac{L}{2} \rfloor,a,b}$, it immediately follows that we get $\hat{f}_{L,a,b} = f_{L,a,b}$ in Step 2(b)ii. The correctness for odd L can be similarly established.

Statement 2. Multiplication of two degree n polynomials can be performed in $O(n \log n)$ using FFT [9]. The degree of polynomials in Steps 2b and 2c are at most the maximum energy for a length ℓ string, $\ell \cdot M$. The total runtime of Step 2b and Step 2c is thus $O(S^5 \ell M \log(\ell M))$. Therefore, the total runtime of Build(ℓ) is $O(\ell(S^5 M \log(\ell M)) + \frac{\ell}{2}(S^5 M \log(\ell M)) + \frac{\ell}{4}(S^5 M \log(\ell M)) + \ldots + 1)$ $= O(S^5 \ell M \log(\ell M)) = O(\ell \log \ell)$. ■

After the execution of Build(L), we have stored a set of approximately $S^5 \log L$ polynomials – $f_{\ell,a,b}(x)$, $f^{d_1,d_2}_{\ell,a,b}(x)$ and $f^{d_1,d_2,d_3}_{\ell,a,b}(x)$. We now show how to quickly extract from these polynomials

Algorithm *Build*(ℓ)

INPUT: Integer ℓ.
OUTPUT: A list of ordered pairs $(X_\ell, Y_\ell), (X_{\lfloor \frac{\ell}{2} \rfloor}, Y_{\lfloor \frac{\ell}{2} \rfloor}), \ldots, (X_1, Y_1)$ such that $X_{\lfloor \frac{\ell}{2^i} \rfloor} = \{f_{\lfloor \frac{\ell}{2^i} \rfloor, a, b} \mid a, b \in \Pi\}$, $Y_{\lfloor \frac{\ell}{2^i} \rfloor} = \{f^{d_1,d_2}_{\lfloor \frac{\ell}{2^i} \rfloor, a, b} \mid a, b, d_1, d_2 \in \Pi\}$ for even $\lfloor \frac{\ell}{2^i} \rfloor$, and $Y_{\lfloor \frac{\ell}{2^i} \rfloor} = \{f^{d_1,d_3,d_2}_{\lfloor \frac{\ell}{2^i} \rfloor, a, b} \mid a, b, d_1, d_2, d_3 \in \Pi\}$ for odd $\lfloor \frac{\ell}{2^i} \rfloor$.

1. If $\ell = 1$, then
   (a) $\hat{f}^{d_1,d_3,d_2}_{\ell,a,b}(x) \leftarrow$ NULL $\forall a, b, d_1, d_2, d_3 \in \Pi$.
   (b) $\hat{f}_{\ell,a,b}(x) \leftarrow \begin{cases} 1 \text{ if } a = b \\ 0 \text{ if } a \neq b \end{cases} \quad \forall a, b \in \Pi$.
   (c) OUTPUT $(\{\hat{f}_{\ell,a,b}(x) \mid a, b \in \Pi\}, \{\hat{f}^{d_1,d_3,d_2}_{\ell,a,b}(x) \mid a, b, d_1, d_2, d_3 \in \Pi\})$.
2. Else ($\ell > 1$)
   (a) LIST $\leftarrow$ Build($\lfloor \frac{\ell}{2} \rfloor$).
   (b) If ℓ is even, then
      i. For all $a, b, d_1, d_2 \in \Pi$, set
      $$\hat{f}^{d_1,d_2}_{\ell,a,b}(x) = \hat{f}_{\lfloor \frac{\ell}{2} \rfloor, a, d_1}(x) \cdot \hat{f}_{\lfloor \frac{\ell}{2} \rfloor, d_2, b}(x) \cdot X^{\Gamma_{d_1,d_2}}.$$
      ii. For all $a, b, \in \Pi$, set
      $$\hat{f}_{\ell,a,b} = \sum_{d_1,d_2 \in \Pi} \hat{f}^{d_1,d_2}_{\ell,a,b}(x).$$
      iii. OUTPUT LIST appended with $(\{\hat{f}_{\ell,a,b}(x) \mid a, b \in \Pi\}, \{\hat{f}^{d_1,d_2}_{\ell,a,b}(x) \mid a, b, d_1, d_2 \in \Pi\})$.
   (c) Else (ℓ is odd)
      i. For all $a, b, d_1, d_2, d_3 \in \Pi$, set
      $$\hat{f}^{d_1,d_2,d_3}_{\ell,a,b}(x) = \hat{f}_{\lfloor \frac{\ell}{2} \rfloor, a, d_1}(x) \cdot \hat{f}_{\lfloor \frac{\ell}{2} \rfloor, d_2, b}(x) \cdot X^{\Gamma_{d_1,d_3}} \cdot X^{\Gamma_{d_3,d_2}}.$$
      ii. For all $a, b, \in \Pi$, set
      $$\hat{f}_{\ell,a,b} = \sum_{d_1,d_2,d_3 \in \Pi} \hat{f}^{d_1,d_2,d_3}_{\ell,a,b}(x).$$
      iii. OUTPUT LIST appended with $(\{\hat{f}_{\ell,a,b}(x) \mid a, b \in \Pi\}, \{\hat{f}^{d_1,d_2}_{\ell,a,b}(x) \mid a, b, d_1, d_2 \in \Pi\})$.

Figure 6: This algorithm builds the polynomials $f_{\ell,a,b}$, $f^{d_1,d_2}_{\ell,a,b}$, and $f^{d_1,d_2,d_3}_{\ell,a,b}$.

Algorithm *Extract*(E, Φ, Ψ)

1. Find $a, b \in \Pi$ such that the coefficient ζ_E in $f_{L,a,b}$ is at least 1. If none exists, output NONE.
2. OUTPUT RecursiveExtract(L,E,a,b).

Algorithm *RecursiveExtract*(ℓ, e, a, b)

1. If $\ell \leq 2$, STOP.
2. Consider the sets $F_a = \{f_{\lfloor \frac{\ell}{2} \rfloor, a, d_1}(x) \mid d_1 \in \Pi\}$ and $F_b = \{f_{\lfloor \frac{\ell}{2} \rfloor, d_2, b}(x) \mid d_2 \in \Pi\}$.
   (a) If ℓ is even, find a pair of polynomials $P = f_{\frac{\ell}{2}, a, d_1}(x) \in F_a$ and $Q = f_{\frac{\ell}{2}, d_2, b}(x) \in F_b$ such that $\zeta^P_{e'}$, $\zeta^Q_{e-e'-\Gamma_{d_1,d_2}} \geq 1$ for some $0 \leq e' \leq \ell M$.
   (b) If ℓ is odd, find a pair of polynomials $P = f_{\lfloor \frac{\ell}{2} \rfloor, a, d_1}(x) \in F_a$ and $Q = f_{\lfloor \frac{\ell}{2} \rfloor, d_2, b}(x) \in F_b$, and $d_3 \in \Pi$ such that $\zeta^P_{e'}$, $\zeta^Q_{e-e'-\Gamma_{d_1,d_3}-\Gamma_{d_3,d_2}} \geq 1$ for some $0 \leq e' \leq \ell M$.
3. $s_1 \leftarrow$ RecursiveExtract$(\lfloor \frac{\ell}{2} \rfloor, e', a, d_1)$.
4. If ℓ is even, OUTPUT $s_1 +$ RecursiveExtract$(\frac{\ell}{2}, e - e' - \Gamma_{d_1,d_2}, d_2, b)$.
5. If ℓ is odd, OUTPUT $s_1 + d_3 +$ RecursiveExtract$(\lfloor \frac{\ell}{2} \rfloor, e - e' - \Gamma_{d_1,d_3} - \Gamma_{d_3,d_2}, d_2, b)$.

Figure 7: Given the output of Build (Figure 6) and/or SlowBuild (Figure 8) as Φ and Ψ respectively, Extract returns an energy E string of length L.

a string of free energy E for any given E, if one exists. Algorithm Extract(E) given in Figure 7 assumes access to the polynomials $f_{\ell,a,b}$ computed by Build(L).

Lemma 7 (Correctness and Time Complexity of Extract(E)).

1. *Given that Build(L) has been run, Extract(E, Φ, Ψ) will return a length L string of free energy E if such a string exists.*
2. *The runtime of Extract(E, Φ, Ψ) is $O(L \log L)$.*

Proof:

Statement 1. To show the correctness of Extract(E, Φ, Ψ), it suffices to show that if there exists a length L string starting with a and ending with b of energy E, then RecursiveExtract(L, E, a, b) will output such a string. More generally, we show that for $\ell_j = \lfloor \frac{L}{2^j} \rfloor$ RecursiveExtract(ℓ_j, e, a, b) outputs an energy e string starting with a and ending with b. Note that this does not hold for arbitrary ℓ as the polynomials $f_{\ell,a,b}$ are only guaranteed to be computed by Build(L) for certain values of ℓ only.

We first note that Step 1 of RecursiveExtract assures that the claim holds for ℓ_j for $j = \lfloor \log L \rfloor$ and $\lfloor \log L \rfloor - 1$. Inductively, assume the claim holds for an arbitrary ℓ_{j+1}, $0 \leq j \leq \lfloor \log L \rfloor - 1$.

Consider the call RecursiveExtract(ℓ_j, e, a, b). Since $\lfloor \frac{\lfloor \frac{\ell}{2^j} \rfloor}{2} \rfloor = \lfloor \frac{\ell}{2^{j+1}} \rfloor$, we know the polynomials $f_{\lfloor \frac{\ell_j}{2} \rfloor, a, b}$ have been computed by Build(L). For the case of even ℓ_j, we know from Step 2 that there exists a length ℓ string of energy e whose first character is a, $\lfloor \frac{\ell}{2} \rfloor^{th}$ character is d_1, $(\lfloor \frac{\ell}{2} \rfloor + 1)^{th}$ character is d_2, and last character is b. By the inductive hypothesis, the recursive calls of Steps 3 and 4 yield the correct $\frac{\ell}{2}$ length strings whose concatenation is the required length ℓ string of energy e starting with a and ending at b. Similarly, for odd ℓ_j we know there exists an energy e string starting with a, ending with b, and containing the three consecutive characters d_1, d_3, d_2 starting at index $\lfloor \frac{\ell}{2} \rfloor$.

Statement 2. To analyze the complexity of Extract(E, Φ, Ψ), first consider the complexity of a call to RecursiveExtract(ℓ, e, a, b). Step 1 can be performed in $O(1)$ time. Step 2(a) can be executed by performing a loop of ℓM iterations for each of the S^2 pairs of characters $d_1, d_2 \in \Pi$ for a complexity of $O(S^2 \ell M)$. Step 2(b) can be executed similarly, except that all triplets $d_1, d_3, d_2 \in \Pi$ need to be considered for a runtime of $O(S^3 \ell M)$.

For each recursive call to RecursiveExtract(ℓ_j, e, a, b), $\ell_j = \lfloor \frac{L}{2^j} \rfloor$, let j denote the recursive depth of the call. For any given depth j, consider the aggregate runtime of all depth j calls to RecursiveExtract. At depth j, the runtime for any single call is bounded by $O(S^3 \ell_j M) = O(S^3 \lfloor \frac{L}{2^j} \rfloor M)$. Further, there are at most 2^{j+1} depth j calls. Thus, the total runtime spent for depth j calls is $O(2^{j+1} S^3 \frac{L}{2^j} M) = O(S^3 L M)$. Therefore, the total runtime for all calls to RecursiveExtract stemming from the initial call from Extract(E, Φ, Ψ), and thus the total runtime of Extract(E, Φ, Ψ), is $O(S^3 L M \log L) = O(L \log L)$. ■

Finally, we describe the remaining subroutine SlowBuild(L) whose purpose is to improve the runtime of Extract(E, Φ, Ψ) from $O(L \log L)$ to $O(L)$ by improving the runtime of RecursiveExtract(ℓ, e, a, b) from $T(\ell) = T(\frac{\ell}{2}) + O(\ell) = O(\ell \log \ell)$ to $T(\ell) = T(\frac{\ell}{2}) + O(1) = O(\ell)$. To do this we decrease the time taken to find each exponent e' in Step 2 of RecursiveExtract(ℓ, e, a, b) from $O(\ell)$, when only Build(L) has been run, to $O(1)$. SlowBuild(L) permits this by explicitly computing each e' for all energies $E \leq \ell M$ and $a, b \in \Pi$. Specifically, SlowBuild solves the following computational problem.

Problem 7 (Exponent Resolution Problem).
INPUT*:*

1. *An integer ℓ;*
2. *For all $a, b, d_1, d_2, d_3 \in \Pi_D$*
   (a) *$f_{\ell,a,b}$,*
   (b) *$f^{d_1,d_2}_{\ell,a,b}$ for even ℓ and $f^{d_1,d_2,d_3}_{\ell,a,b}$ for odd ℓ, and*
   (c) *$f_{\lfloor \ell/2 \rfloor,a,b}$.*

OUTPUT*: For all $a, b \in \Pi_D$ and even ℓ, a length $\ell \cdot M$ array $E^{\ell}_{a,b}$ where $E^{\ell}_{a,b}[i] = (d_1, d_2, e')$ for $0 \leq i \leq \ell \cdot M$ where the coefficient of $x^{e'}$ in $f_{\lfloor \ell/2 \rfloor,a,d_1}$ is non-zero and the coefficient of $x^{i-e'-\Gamma_{d_1,d_2}}$ in $f_{\lfloor \ell/2 \rfloor,d_2,b}$ is non-zero. Similarly, for all $a, b \in \Pi_D$ and odd ℓ, a length $\ell \cdot M$ array $E^{\ell}_{a,b}$ where $E^{\ell}_{a,b}[i] = (d_1, d_2, d_3, e')$ for $0 \leq i \leq \ell \cdot M$ where the coefficient of $x^{e'}$ in $f_{\lfloor \ell/2 \rfloor,a,d_1}$ is not 0 and the coefficient of $x^{i-e'-\Gamma_{d_1,d_3}-\Gamma_{d_3,d_2}}$ in $f_{\lfloor \ell/2 \rfloor,d_2,b}$ is not 0.*

We now show the correctness and time complexity of SlowBuild(L) given in Figure 8.

Algorithm $SlowBuild(L)$

1. For $i \leftarrow 0$ to $\lfloor \log L \rfloor$, OUTPUT SlowBuildHelper$(\lfloor L/2^i \rfloor, X_{\lfloor \frac{L}{2^i} \rfloor}, Y_{\lfloor \frac{L}{2^i} \rfloor}, X_{\lfloor \frac{L}{2^{i+1}} \rfloor})$.

Algorithm $SlowBuildHelper(\ell, X_\ell, Y_\ell, X_{\ell/2})$

1. For each $i \in [0, \ell \cdot M], a, b \in \Pi$ such that the coefficient of x^i is 0 in $f_{\ell,a,b}$, set $E^\ell_{a,b}[i] = \text{NULL}$.
2. For each $i \in [0, \ell \cdot M], a, b \in \Pi$ such that the coefficient of x^i is non-zero in $f_{\ell,a,b}$, do the following
   (a) If ℓ is even,
      i. For all $d_1, d_2 \in \Pi$, set $\mathcal{C}_{d_1,d_2} = \emptyset$.
      ii. find d_1, d_2 such that the coefficient of x^i in $f^{d_1,d_2}_{\ell,a,b} \neq 0$. Let $\mathcal{C}_{d_1,d_2} = \mathcal{C}_{d_1,d_2} \cup \{i - \Gamma_{d_1,d_2}\}$.
      iii. $\mathcal{D}_{d_1,d_2} \leftarrow$ DeduceFactors$(\mathcal{C}_{d_1,d_2}, f_{\lfloor \frac{\ell}{2} \rfloor,a,d_1}(x), f_{\lfloor \frac{\ell}{2} \rfloor,d_2,b}(x), \ell)$.
      iv. For all $(i,j) \in \mathcal{D}_{d_1,d_2}$, $E^\ell_{a,b}[i + \Gamma_{d_1,d_2}] \leftarrow (d_1, d_2, j)$.
   (b) If ℓ is odd,
      i. For all $d_1, d_2, d_3 \in \Pi$, set $\mathcal{C}_{d_1,d_2,d_3} = \emptyset$.
      ii. find d_1, d_2, d_3 such that the coefficient of x^i in $f^{d_1,d_2,d_3}_{\ell,a,b} \neq 0$. Let $\mathcal{C}_{d_1,d_2} = \mathcal{C}_{d_1,d_2,d_3} \cup \{i - \Gamma_{d_1,d_3} - \Gamma_{d_3,d_2}\}$.
      iii. $\mathcal{D}_{d_1,d_2,d_3} \leftarrow$ DeduceFactors$(\mathcal{C}_{d_1,d_2,d_3}, f_{\lfloor \frac{\ell}{2} \rfloor,a,d_1}(x), f_{\lfloor \frac{\ell}{2} \rfloor,d_2,b}(x), \ell)$.
      iv. For all $(i,j) \in \mathcal{D}_{d_1,d_2,d_3}$, $E^\ell_{a,b}[i + \Gamma_{d_1,d_3} + \Gamma_{d_3,d_2}] \leftarrow (d_1, d_2, d_3, j)$.
3. For each $a, b \in \Pi$, OUTPUT $E^\ell_{a,b}$.

Figure 8: This algorithm solves the Exponent Resolution Problem (Problem 7).

Algorithm $DeduceFactors(\mathcal{C}, g(x), h(x), \ell)$

1. If $\mathcal{C} = \emptyset$, return $\emptyset$.
2. Let $m \leftarrow \max\{\deg\, g(x), \deg\, f(x)\}$.
3. If $m \leq \sqrt{\ell \log \ell}$, then
   (a) $\mathcal{D} \leftarrow \emptyset$.
   (b) For every $i \in \mathcal{C}$, let $\mathcal{D} \leftarrow \mathcal{D} \cup \{(i, j)\}$ for some j such that the coefficient of x^j in $g(x)$ and that of x^{i-j} in $h(x)$ is not 0.
   (c) Return $\mathcal{D}$.
4. Let $g_0(x)$ and $g_1(x)$ be degree at most $\lfloor m/2 \rfloor$ polynomials such that $g(x) = g_0(x) + g_1(x) \cdot x^{\lfloor m/2 \rfloor + 1}$. Similarly, let $h_0(x)$ and $h_1(x)$ be degree $\lfloor m/2 \rfloor$ polynomials such that $h(x) = h_0(x) + h_1(x) \cdot x^{\lfloor m/2 \rfloor + 1}$.
5. Let $f_{00} \leftarrow g_0 \cdot h_0$, $f_{01} \leftarrow g_0 \cdot h_1$, $f_{10} \leftarrow g_1 \cdot h_0$, and $f_{11} \leftarrow g_1 \cdot h_1$.
6. Let $\mathcal{C}_{00} \leftarrow \emptyset$,$\mathcal{C}_{01} \leftarrow \emptyset$,$\mathcal{C}_{10} \leftarrow \emptyset$, and $\mathcal{C}_{11} \leftarrow \emptyset$.
7. For all $i \in \mathcal{C}$
   (a) If the coefficient of x^i in f_{00} is not 0, then $\mathcal{C}_{00} \leftarrow \mathcal{C}_{00} \cup \{i\}$,
   (b) Else if the coefficient of $x^{i-(\lfloor m/2 \rfloor + 1)}$ in f_{01} is not 0, then $\mathcal{C}_{01} \leftarrow \mathcal{C}_{01} \cup \{i - (\lfloor m/2 \rfloor + 1)\}$.
   (c) Else if the coefficient of $x^{i-(\lfloor m/2 \rfloor + 1)}$ in f_{10} is not 0, then $\mathcal{C}_{10} \leftarrow \mathcal{C}_{10} \cup \{i - (\lfloor m/2 \rfloor + 1)\}$.
   (d) Else if the coefficient of $x^{i-2(\lfloor m/2 \rfloor + 1)}$ in f_{11} is not 0, then $\mathcal{C}_{11} \leftarrow \mathcal{C}_{11} \cup \{i - 2(\lfloor m/2 \rfloor + 1)\}$.
8. Let $\mathcal{D}_{00} = \text{DeduceFactors}(\mathcal{C}_{00}, g_0, h_0, \ell)$.
   Let $\mathcal{D}_{01} = \text{DeduceFactors}(\mathcal{C}_{01}, g_0, h_1, \ell)$.
   Let $\mathcal{D}_{10} = \text{DeduceFactors}(\mathcal{C}_{10}, g_1, h_0, \ell)$.
   Let $\mathcal{D}_{11} = \text{DeduceFactors}(\mathcal{C}_{11}, g_1, h_1, \ell)$.
9. $\mathcal{D} \leftarrow \mathcal{D}_{00}$.
10. For all $(i, j) \in \mathcal{D}_{01}$, let $\mathcal{D} = \mathcal{D} \cup \{(i + \lfloor m/2 \rfloor + 1, j)\}$.
11. For all $(i, j) \in \mathcal{D}_{10}$, let $\mathcal{D} = \mathcal{D} \cup \{(i + \lfloor m/2 \rfloor + 1, j + \lfloor m/2 \rfloor + 1)\}$.
12. For all $(i, j) \in \mathcal{D}_{11}$, let $\mathcal{D} = \mathcal{D} \cup \{(i + 2(\lfloor m/2 \rfloor + 1), j + \lfloor m/2 \rfloor + 1)\}$.
13. Return $\mathcal{D}$.

Figure 9: For all $i \in \mathcal{C}$ such that the coefficient of x^i in $g(x) \cdot h(x)$ is non-zero, DeduceFactors($\mathcal{C}$, $g(x)$, $h(x)$, ℓ) returns a set $\mathcal{D}$ containing (i, j) such that the coefficient of x^j in $g(x)$ and that of x^{i-j} in $h(x)$ are non-zero.

Lemma 8 (Correctness and Time Complexity of SlowBuild(L)).

1. *SlowBuild(L) solves Problem 7 for each $\ell = \lfloor \frac{L}{2^j} \rfloor$.*
2. *The runtime of SlowBuild(L) is $O(L^{1.5} \log^{0.5} L)$.*

PROOF:

Statement 1. To show this we must show that SlowBuildHelper(ℓ) solves Problem 7 for any $\ell = \lfloor \frac{L}{2^j} \rfloor$. This follows from the following claim.

Claim 1. *For all $i \in \mathcal{C}$ such that the coefficient of x^i in $g(x) \cdot h(x)$ is non-zero, the set $\mathcal{D} =$ DeduceFactors($\mathcal{C}$, $g(x)$, $h(x)$, ℓ) contains (i, j) such that the coefficient of x^j in $g(x)$ and that of x^{i-j} in $h(x)$ are non-zero.*

PROOF: Let $m = \max\{\deg g(x), \deg f(x)\}$. We inductively show that the claim holds for all positive values of m. As a base case, if $m \leq \sqrt{\ell \log \ell}$, then the claim holds by Step 2 of the algorithm. As the inductive step, consider a call to DeduceFactors($\mathcal{C}, g(x), h(x), \ell$) such that $m \geq \sqrt{\ell \log \ell}$. Inductively assume the claim holds for all $m' \leq \lfloor \frac{m}{2} \rfloor$.

Now consider an arbitrary term x^i in $g(x)h(x)$ such that x^i has a non-zero coefficient. We need to show that the routine will output a D that contains an appropriate (i, j) as defined by problem 7. Since

$$g(x)h(x) = g_0(x)h_0(x) + g_0(x)h_1(x)x^{\lfloor \frac{m}{2} \rfloor + 1} + g_1(x)h_0(x)x^{\lfloor \frac{m}{2} \rfloor + 1} + g_1(x)h_1(x)x^{2(\lfloor \frac{m}{2} \rfloor + 1)}$$

we know that either the term x^i has a non-zero coefficient in $g_0(x)h_0(x)$, or $x^{i-\lfloor \frac{m}{2} \rfloor + 1}$ has a non-zero coefficient in $g_0(x)h_1(x)$, or $x^{i-\lfloor \frac{m}{2} \rfloor + 1}$ has a non-zero coefficient in $g_1(x)h_0(x)$, or $x^{i-2(\lfloor \frac{m}{2} \rfloor + 1)}$ has a non-zero coefficient in $g_1(x)h_1(x)$. To finish the proof, we consider these four cases separately.
Case 1: x^i has a non-zero coefficient in $g_0(x)h_0(x)$. In this case, by inductive hypothesis, Step 8 ensures that there exists $(i, j) \in D_{00} \subseteq D$ such that the terms x^j in $g_0(x)$ and x^{i-j} in $h_0(x)$ are non-zero. This immediately implies that x^j and x^{i-j} in $g(x)h(x)$ are non-zero.
Case 2: $x^{i-\lfloor \frac{m}{2} \rfloor - 1}$ has a non-zero coefficient in $g_0(x)h_1(x)$. In this case, if the requirements for Case 1 are not satisfied, we have from Step 8 that there is an $(i - \lfloor \frac{m}{2} \rfloor - 1, j) \in D_{0,1}$ such that x^j has a non-zero coefficient in $g_0(x)$ and $x^{i-\lfloor \frac{m}{2} \rfloor - 1 - j}$ has a non-zero coefficient in $h_1(x)$. The x^j term is thus non-zero in $g(x)$ and x^{i-j} has a non-zero coefficient in $h_1(x)x^{\lfloor \frac{m}{2} \rfloor + 1}$, implying it has a non-zero coefficient in $h(x)$. In Step 10 places the pair (i, j) into D.
Case 3: $x^{i-\lfloor \frac{m}{2} \rfloor - 1}$ has a non-zero coefficient in $g_1(x)h_0(x)$. In this case, if the requirements for Cases 1 and 2 are not satisfied, we know that there exists a pair $(i - \lfloor \frac{m}{2} \rfloor - 1, j') \in D_{1,0}$ such that $x^{j'}$ in $g_1(x)$ and $x^{i-\lfloor \frac{m}{2} \rfloor - 1 - j'}$ in $h_0(x)$ are non-zero. Thus, $x^j = x^{j' + \lfloor \frac{m}{2} \rfloor + 1}$ has a non-zero coefficient in $g(x)$ and $x^{i-\lfloor \frac{m}{2} \rfloor - 1 - j'} = x^{i-j}$ has a non-zero coefficient in $h(x)$. In Step 11 the pair (i, j) is added to D.
Case 4: $x^{i-2(\lfloor \frac{m}{2} \rfloor + 1)}$ has a non-zero coefficient in $g_1(x)h_1(x)$. In this case, if the requirements for Cases 2 and 3 are not satisfied, we have from Step 8 that there is an $(i - 2(\lfloor \frac{m}{2} \rfloor + 1), j')$ pair in $D_{1,1}$ such that $x^j = x^{j' + \lfloor \frac{m}{2} \rfloor + 1}$ has a non-zero coefficient in $g(x)$ and $x^{i-(\lfloor \frac{m}{2} \rfloor + 1) - j'} = x^{i-j}$ has a non-zero coefficient in $h(x)$. In Step 12 the pair (i, j) is added to D. ■

Statement 2. SlowBuildHelper(ℓ) makes at most S^5 calls to DeduceFactors($\mathcal{C}$, $g(x)$, $h(x)$, ℓ) with $|\mathcal{C}| \leq \ell$ and $\max\{\deg g(x), \deg f(x)\} \leq \ell \cdot M/2 = O(\ell)$. We will show that each of these calls take time $O(\ell\sqrt{\ell \log \ell})$. Let $T(m)$ be the running time of DeduceFactors($\mathcal{C}$, $g(x)$, $h(x)$, ℓ) where

$m = \max\{\deg\, g(x),\, \deg\, f(x)\}$. Using FFT for polynomial multiplication we have the following recurrence relation for $T(m)$.

$$T(m) = 4T(m/2) + O(m \log m).$$

Solving this recurrence we get $T(m) = 4^k T(\frac{m}{2^k}) + O(m(\log m + 2\log(m/2) + \cdots + 2^{k-1}\log(m/2^{k-1}))) = 4^k T(\sqrt{m \log m}) + O(m\sqrt{m \log m})$. Note that for $m \leq \sqrt{\ell \log \ell}$, $T(m) = |\mathcal{C}| \cdot m$. Since the total number of elements in all the sets $\mathcal{C}$ at any level of the recursion tree does not increase, the total time taken for solving all calls to DeduceFactors with $m \leq \sqrt{\ell \log \ell}$ is $O(\ell\sqrt{\ell \log \ell})$. Therefore, $T(\ell) = 4^k T(\sqrt{\ell \log \ell}) + O(\ell\sqrt{\ell \log \ell}) = O(\ell\sqrt{\ell \log \ell})$. Since SlowBuildHelper($\ell$) makes at most S^5 calls to DeduceFactors($\mathcal{C}, g(x), h(x), \ell$), the runtime of SlowBuildHelper(ℓ) is $O(S^5 \ell\sqrt{\ell \log \ell})$ and that of SlowBuild(L) is $O(S^5(L\sqrt{L \log L} + L/2\sqrt{L/2 \log L/2} + \cdots)) = O(S^5 L\sqrt{L \log L}) = O(L\sqrt{L \log L})$. ■

Lemma 9 (Improved runtime of Extract(E)). *If SlowBuild(L) has been executed, Extract(E) can be implemented to run in time $O(L)$.*

Proof: After the execution of SlowBuild(L), from Lemma 8 each array $E^{\ell}_{a,b}$ has been filled for each $l = \lfloor \frac{L}{2^i} \rfloor$, $i = 0, \ldots \lfloor \log L \rfloor$. Thus, for even ℓ, in time $O(1)$ we can produce the d_1, d_2, and e' for Step 2 of RecursiveExtract by retrieving the value $E^{\ell}_{a,b}[e]$ computed by SlowBuild. Similarly we can obtain d_1, d_2, d_3, and e' for odd ℓ in time $O(1)$. Thus, the total runtime of RecursiveExtract for a given parameter ℓ is $2T(\frac{\ell}{2}) + O(1) = O(\ell)$. The runtime of Extract(L) is thus $O(L)$. ■

We now apply Lemmas 6, 7, 8, and 9, to prove the following theorem.

Theorem 5. *Algorithm ConstructStrings($\{A_i\}, \{B_i\}, L$) solves Problem 6 in time $O(\min\{nL \log L, L^{1.5} \log^{0.5} L + nL\})$.*

Proof: For correctness, since ConstructStrings($\{A_i\}, \{B_i\}, L$) calls Build(L) in step 1, it follows from Lemma 7 that Step 4 outputs the solution to Problem 6. For time complexity, Step 1 takes $O(L \log L)$ by Lemma 6. Step 3 takes $O(n\Delta) = O(n)$. For Steps 2 and 4, consider the two cases for the value of n with respect to L.

Case 1: $n \geq \sqrt{\frac{L}{\log L}}$. In this case, Step 2 takes $O(L^{1.5} \log^{0.5} L)$ by Lemma 8 and Step 4 takes $O(nL)$ by Lemma 9. We thus have a time complexity of $O(L^{1.5} \log^{0.5} L + nL) = O(L^{1.5} \log^{0.5} L)$. We also have that $\min\{nL \log L,\, L^{1.5} \log^{0.5} L + nL\} \geq L^{1.5} \log^{0.5} L$, implying that the time complexity of the algorithm is $O(\min\{nL \log L,\, L^{1.5} \log^{0.5} L + nL\})$.

Case 2: $n < \sqrt{\frac{L}{\log L}}$. In this case, Step 2 takes $O(1)$ time and Step 4 takes $O(nL \log L)$ by Lemma 7. We have that $\min\{nL \log L,\, L^{1.5} \log^{0.5} L + nL\} = nL \log L$. This implies that the time complexity of the algorithm is $O(\min\{nL \log L,\, L^{1.5} \log^{0.5} L + nL\})$. ■

4.3 Putting it all together

Theorem 6. *Algorithm* FastDWD$_{1,2,3,4,5,8}$ *produces a set of n words of length $\ell = \Theta(k + \log n)$ in time $O(\min\{n\ell \log \ell, \ell^{1.5} \log^{0.5} \ell + n\ell\})$ satisfying the constraints $C_1(k_1)$, $C_2(k_2)$, $C_3(k_3)$, $C_4(k_4)$, $C_5(k_5)$, and $C_8(4D + \Gamma_{\max})$ with probability of failure at most $o(1/(n + 2^k))$, where $k = \max\{k_1, k_2, k_3, k_4, k_5\}$.*

Proof: From Theorem 2 we know that $\mathcal{W}$ satisfies constraints $C_1(k_1)$, $C_2(k_2)$, $C_3(k_3)$, $C_4(k_4)$ and $C_5(k_5)$ with probability of failure at most $o(1/(n + 2^k))$. If $FE(S_b) - FE(S_a) \leq 3D$, then Algorithm FastDWD$_{1,2,3,4,5,8}$ outputs $\mathcal{W}$ which satisfies $C_8(3D)$ and hence also satisfies $C_8(4D +$

$\Gamma_{\max}$). Otherwise, it is easy to verify that since $\mathcal{W}$ satisfies these 5 constraints, so does $\mathcal{W}'$. Also, the length of any word in $W' \in \mathcal{W}'$ is at most 3ℓ where $\ell = \Theta(k + \log n)$. From Lemma 3 we know that there always exists a string S'_j as required in Step 4 of Algorithm FastDWD$_{1,2,3,4,5,8}$. Further, Lemma 4 shows that $\mathcal{W}'$ satisfies $C_8(\Delta + 2D + \Gamma_{\max})$.

Generating $\mathcal{W}$ takes $O(n \cdot k + n \cdot \log n)$ time. The bulk of the time complexity for the algorithm comes from Step 4 which is analyzed in Section 4.2 to get $O(\min\{nL \log L, L^{1.5} \log^{0.5} L + nL\})$ (Theorem 5) where $L = O(\ell)$. ■

5 Future Work

There are a number of problems related to this work that remain open. For example, it is still unknown how to generate codewords of optimal length that simultaneously satisfy the free energy constraint and the consecutive bases constraint. We also have not provided a method for combining the consecutive bases constraint with the shifting Hamming constraint.

Another open research area is the problem of verifying whether or not a set of codewords satisfy a given set of constraints. This problem is important because our algorithms only provide a high probability assurance of success. While verification can clearly be done in polynomial time for all of our constraints, the naive method of verification has a longer runtime than our algorithms for constructing the sets. Finding faster, non-trivial verification algorithms is an open problem.

A third direction for future work involves considering a generalized form of the Hamming constraint. There are applications in which it is desirable to design sets of strands such that some distinct pairs bind with one another, while others do not [2, 15]. In this scenario, we can formulate a word design problem that takes as input a matrix of pairwise requirements for Hamming distances. Determining when such a problem is solvable, and how to solve it optimally when it is are open problems.

References

[1] L. M. Adleman, *Molecular Computation of Solutions to Combinatorial Problems*, Science, 266 (1994), pp. 1021–1024.

[2] G. Aggarwal, M. H. Goldwasser, M.-Y. Kao, and R. T. Schweller, *Complexities for Generalized Models of Self-Assembly*, in Proceedings of the 15th Annual ACM-SIAM Symposium on Discrete Algorithms, Society for Industrial and Applied Mathematics, 2004, pp. 880–889.

[3] A. Ben-Dor, R. Karp, B. Schiwkowski, and Z. Yakhini, *Universal DNA Tag Systems: A Combinatorial Design Scheme*, in Proceedings of the 4th Annual International Conference on Computational Molecular Biology, 2000, pp. 65–75.

[4] R. S. Braich, C. Johnson, P. W. K. Rothemund, D. Hwang, N. V. Chelyapov, and L. M. Adleman, *Solution of a Satisfiability Problem on a Gel-Based DNA Computer*, in DNA '00: Revised Papers from the 6th International Workshop on DNA-Based Computers, Springer-Verlag, 2001, pp. 27–42.

[5] S. Brenner, *Methods for Sorting Polynucleotides using Oligonucleotide Tags.* US Patent Number 5,604,097, February 1997.

[6] S. Brenner and R. A. Lerner, *Encoded Combinatorial Chemistry*, in Proceedings of Natianal Academy of Science, vol. 89, June 1992, pp. 5381–5383.

[7] A. Brennerman and A. E. Condon, *Strand Design for Bio-Molecular Computation*, Theoretical Computer Science, 287 (2001), pp. 39–58.

[8] K. J. Breslauer, R. Frank, H. Blocker, and L. A. Marky, *Predicting DNA Duplex Stability from the Base Sequence* , in Proceedings of the National Academy of Sciences, vol. 83, 1986, pp. 3746–3750.

[9] T. H. Cormen, C. L. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, MIT Press, Cambridge, MA, 2nd ed., 2001.

[10] R. Deaton, M. Garzon, R. Murphy, D. Franceschetti, and S. Stevens, *Genetic Search of Reliable Encodings for DNA Based Computation*, in Proceedings of the 1st Annual Conference on Genetic Programming, 1996.

[11] A. G. Frutos, Q. Liu, A. J. Thiel, A. M. W. Sanner, A. E. Condon, L. M. Smith, and R. M. Corn, *Demonstration of a Word Design Strategy for DNA Computing on Surfaces*, Nucleic Acids Research, 25 (1997), pp. 4748–4757.

[12] M. Garzon, R. Deaton, P. Neathery, D. Franceschetti, and R. Murphy, *A New Metric for DNA Computing*, in Proceedings of the 2nd Genetic Programming Conference, 1997, pp. 472–278.

[13] A. Marathe, A. Condon, and R. M. Corn, *On Combinatorial DNA Word Design.*, Journal of Computational Biology, 8 (2001), pp. 201–219.

[14] D. D. Shoemaker, D. A. Lashkari, D. Morris, M. Mittmann, and R. W. Davis, *Quantitative Phenotypic Analysis of Yeast Deletion Mutants Using a Highly Parallel Molecular Bar-coding Strategy*, Nature, 16 (1996), pp. 450–456.

[15] S. A. Tsaftaris, *DNA Computing from a Signal Processing Viewpoint*, IEEE Signal Processing Magazine, 21 (2004), pp. 100–106.

[16] D. C. Tulpan and H. H. Hoos, *Hybrid Randomised Neighbourhoods Improve Stochastic Local Search for DNA Code Design.*, in Canadian Conference on AI, 2003, pp. 418–433.

[17] D. C. Tulpan, H. H. Hoos, and A. Condon, *Stochastic Local Search Algorithms for DNA Word Design*, in DNA, March 2002, pp. 229–241.

[18] E. Winfree, F. Liu, L. Wenzler, and N. Seeman, *Design and Self-Assembly of Two-Dimensional DNA Crystals*, Nature, 394 (1998), pp. 539–544.